

OBSERVAÇÕES:

1) Prova individual.

2) O valor final da avaliação será proporcional a 5,0.

1. (1,0) Sobre os conceitos de processo e programa, julgue os itens como verdadeiro (v) ou falso (f).

(f) Os conceitos de programa e processo podem ser considerados sinônimos.

(v) Um programa determina o conjunto de instruções, em uma linguagem de alto nível, que o programador deseja que o computador execute.

(v) Para que um programa seja útil, este deve ser “traduzido” para um formato compreensível pelo computador, de modo que possa ser executado.

(v) Pode-se dizer que o processo tem o papel de agregador de recursos que serão utilizados quando este executar as funções para as quais foi programado.

(f) Processos podem executar ações concorrentemente e em paralelo apenas mediante o uso de IPC.
2. (1,4) Sobre o famoso debate realizado entre A. S. Tanenbaum e Linus Torvalds no início da década de 90 (fonte: <https://www.oreilly.com/openbook/opensources/book/appa.html>), julgue os itens a seguir (v para verdadeiro e f para falso).

(v) Sistemas operacionais antigos, via de regra, são monolíticos, significando que o SO como um todo é um único arquivo binário que é executado em modo kernel. Este binário contém o gerenciamento de processos, gerenciamento de memória, gerenciamento de sistema de arquivo etc. Exemplos de sistemas assim são o UNIX, o MS-DOS, o MULTICS entre outros.

(f) Em sistemas baseados em microkernel, a maior parte do SO é executada como processos integrados, via de regra, internos ao kernel “principal”. A tarefa do kernel é lidar com a comunicação entre os processos via chamadas de função, o que permite o gerenciamento de interrupções, gerenciamento de processo de baixo nível, e possivelmente E/S. Exemplos de sistemas nesta categoria são o RC4000, Amoeba, Chorus, Mach e o Windows/NT.

(f) A. S. Tanenbaum é defensor de sistemas baseados em kernels monolíticos e destaca suas características principais de projeto, principalmente com relação a modularidade e portabilidade.

(v) Um quesito que poderia ser pontuado para sistemas monolíticos seria o desempenho, mas de acordo com A. S. Tanenbaum, esse quesito inclusive já foi comprovado como sendo equivalente em sistemas baseados em microkernels.

(f) Linux Torvalds destaca que os benefícios de plataformas baseadas em microkernels são principalmente estéticos. Além disso, quando Torvalds lançou o Minix, não haviam opções baseadas em microkernel equivalentes (e.g. GNU Hurd) disponíveis.

(v) Torvalds alega que o Minix, criação de A. S. Tanenbaum não está livremente disponível e, além disso, a implementação de microkernel do Minix é ruim (gerenciamento ruim de passagem de mensagens e multi-tarefa).

(v) Outro ponto defendido por Torvalds diz respeito a portabilidade do Linux frente ao Minix, por exemplo. Como o código do Linux é padronizado (o mais próximo possível do POSIX), portar códigos para o Linux é mais simples do que portá-los para o MINIX.

3. (1,0) Analise o seguinte trecho de código:

...

```
1. pid t pid;  
2. pid = fork();  
3. if (pid == 0) { /* processo filho */  
4. fork();  
5. thread create(...);  
6. }  
7. fork();
```

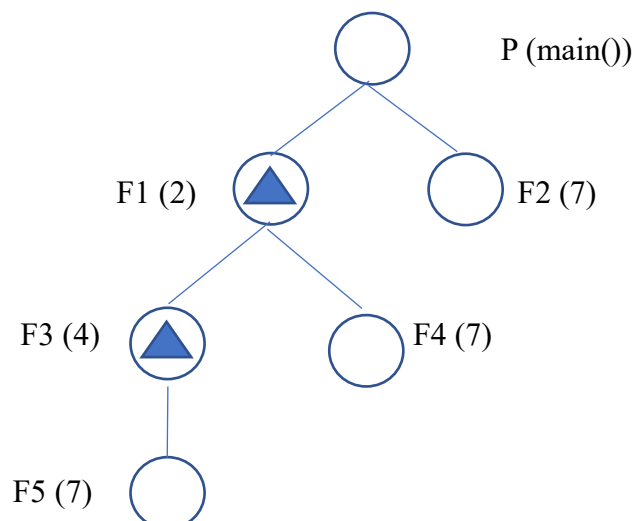
a) (0,5) Quantos processos são criados (incluindo o processo pai)?

Resposta: 6 processos

b) (0,5) Quantas *threads* são criadas via *thread create(...)*?

Resposta: 2 *threads*

Representação: círculo = processo e triângulo = *thread*.



4. (0,6) Explique o que é uma chamada de sistema (*system call*) e qual é o papel desempenhado pelas APIs (Application Program Interfaces) nesse contexto.

Resposta: As chamadas de sistema (*system calls*) representam a interface de programação para os serviços oferecidos pelo SO. O acesso para essas interfaces é tipicamente realizado via linguagem de alto nível (ex. C, C++ etc), via interface de programação de alto nível (API) ao invés do uso direto da chamada de sistema.

5. (1,0) Considere o seguinte trecho da *man page* para as chamadas de sistema da família `exec()`:

*“A família de funções **exec** substitui o processo corrente com uma nova imagem de processo. As funções descritas nesse manual são front-ends para a função `execve(2)`*

...

*As funções **execv()**, **execvp()**, e **execvP()** oferecem um vetor de ponteiros para strings terminadas em `NULL` que representam a lista de argumentos disponíveis ao novo programa. O primeiro argumento, por convenção, deve apontar para o arquivo associado com o executável desejado. O vetor de ponteiros **deve** ser terminado pelo ponteiro `NULL`.*

...

*As funções **execvp()**, **execvp()**, e **execvP()** duplicarão as ações do shell ao procurar por um arquivo executável se o nome especificado não contiver um caracter barra “/”. Para o **execvp()** e **execvp()**, o caminho de busca é o caminho especificado na variável de ambiente “`PATH`”...*

Agora considere o trecho de código abaixo.

```
char *argv[2] = {"", NULL};
int pid = fork();
    if ( pid == 0 ) {
        execvp("ls", argv);
        thread create(...);
    }
...
```

- a) (0,5) Considerando o que foi exposto, o que as chamadas de sistema `fork()` e `execvp()` desempenham em conjunto nesse trecho de código?

Resposta: `fork()` irá criar um processo filho e o processo filho será substituído pelo comando “ls” quando a chamada de sistema for executada em seu contexto. Como `execvp` possui o caminho de busca definido na variável de ambiente `PATH`, ela terá condições de encontrar o comando “ls”.

- b) (0,5) Quantas *threads* são criadas? Justifique sua resposta.

Resposta: Nenhuma *thread* será criada. Quando a chamada de sistema `execvp()` for executada, o processo filho será substituído pelo comando “ls” e a linha da instrução de criação de *thread* nunca será executada.

6. (1,0) Considere o conceito de processos leves (LightWeight Process) considere também que um processo de espaço de usuário que use *threads*, executa sua *thread* sobre um LWP e este é atrelado a uma *thread* de kernel, a qual executa nos processadores físicos. Quando uma *thread* está prestes a bloquear, o kernel realiza um *upcall* à aplicação, informando-a que uma *thread* está prestes a bloquear e identifica a *thread* específica. Em seguida, o kernel aloca um novo LWP à aplicação. A aplicação executa um *upcall handler* neste novo LWP, o qual salva o estado da *thread* bloqueante e encerra (libera) o LWP que executa a *thread* bloqueante. Em seguida, o *upcall handler* escalona outra *thread* que tem condições de ser executada no novo LWP. Pergunta-se: sob a ótica de sistemas de tempo real, essa política pode ser prejudicial? Explique.

Resposta: Potencialmente sim. Tempo é um quesito crucial em aplicações de tempo real. Se uma *thread* é marcada como de tempo real, mas não é atrelada a um LWP, a *thread* pode ter que esperar para ser atrelada a um LWP antes de ser executada. Considere que uma *thread* de tempo real está em execução (é atrelada a um LWP) e então é bloqueada (i.e. deve realizar I/O, foi interrompida por uma *thread* de tempo real de maior prioridade, está esperando por um lock de exclusão mútua etc). Enquanto a *thread* de tempo real está bloqueada, o LWP para o qual foi atrelada pode ser atribuído a outra *thread*. Quando a *thread* de tempo real foi escalonada para executar novamente, ela deve primeiro esperar para ser atrelada a um LWP. Para evitar este tempo de espera, seria interessante a ligação de um LWP a uma *thread* de tempo real durante a sua existência, de modo a assegurar que esta estará apta a executar com retardo (*delay*) mínimo uma vez que for escalonada.

7. (1,0) Considerando IPC, julgue os seguintes itens (v para verdadeiro e f para falso).

(v) Socket consiste na concatenação de informações da camada de rede e da camada de transporte (endereço IP e porta). Por exemplo, o *socket* 161.25.19.8:1625 refere-se a porta 1625 no host 161.25.19.8. A comunicação é descrita por um par de sockets e pelo protocolo de transporte. Ex. TCP: 161.25.19.8:1625 → 146.86.5.20:80, identificando unicamente essa comunicação na Internet.

(v) PIPEs são arquivos especiais no sistema de arquivos. Podem funcionar em modo simplex, half-duplex ou full-duplex dependendo do tipo usado. Via de regra, permite que dois processos locais (i.e. compartilhando um mesmo sistema de arquivos) troquem informações.

Considerando o seguinte programa:

```
#define HALF_DUPLEX    "/tmp/halfduplex"
#define MAX_BUF_SIZE   255
#define NFORKS 2

typedef struct {
    int            id;
    char           data[MAX_BUF_SIZE];
```

```

} PACKET;

int
main(int argc, char *argv[])
{
    pid_t            whichone, reader, w1;
    int              status, howmany, fd, ret_val, count,
numread;
    PACKET           r, p1;

    if ((reader = fork()) == 0) {

        ret_val = mkfifo(HALF_DUPLEX, 0666);

        if ((ret_val == -1) && (errno != EEXIST)) {
            perror("\nError creating the named pipe");
            exit(1);
        }

        fd = open(HALF_DUPLEX, O_RDONLY);

        read(fd, &r, sizeof(r));
        printf("\n====\n");
        printf("ID: %d\n", r.id);
        printf("MSG: %s\n", r.data);
        printf("\n====\n");
        exit(0);
    } else if ((w1 = fork()) == 0) {
        sleep(1);
        fd = open(HALF_DUPLEX, O_WRONLY);
        p1.id = getpid();
        sprintf(p1.data, "%s %d", "Hiya! My ID is",
getpid());
        write(fd, &p1, sizeof(p1));
        exit(0);
    }

    howmany = 0;
    while (howmany < NFORKS) {

```

```

        whichone = wait(&status);
        howmany++;
        printf("\nExited (%d).\n", whichone);
    }
}

```

(v) É possível identificar a criação de 3 processos. O processo pai e dois processos filhos.

(f) Os processos filhos estão se comunicando via *socket*. O processo *w1* abre o *socket* “/tmp/halfduplex” para escrita e insere uma estrutura *PACKET* preenchida com seu ID e uma mensagem. O processo *reader* abre o *socket* “/tmp/halfduplex” para leitura e imprime os dados contidos em *PACKET* (provenientes de *w1*).

(v) Transmite-se no canal uma struct (com alias *PACKET*) que foi preenchida com o ID de um processo *w1* e com uma mensagem *Hiya! My ID is <ID>*, com *<ID>* sendo preenchido com o id de *w1*.

8. (2,0) Considere o código abaixo.

```

#define TRUE 1
#define FALSE 0

pthread_t tid[2];

volatile int counter = 0, turn = 0, flag[2]={FALSE, FALSE};

void
enterSC (int i)
{
    int j=1-i;
    flag[i] = TRUE;
    turn = j;
    while(flag[j] && turn == j);
    return;
}

void
leaveSC (int i)
{
    flag[i] = FALSE;

    return;
}

```

```

void * p (void *arg){
    int id = (int *)arg;

    if (id)
        printf("\nPj: A região crítica é minha!!!\n");
    else
        printf("\nPi: A região crítica é minha!!!\n");
    counter++;
    if (id)
        printf("\n\nPj: counter = %d\n\n", counter);
    else
        printf("\n\nPi: counter = %d\n\n", counter);

    return NULL;
}

int main (void)
{
    int err;
    pthread_create (&(tid[0]), NULL, p, (void *)0);
    pthread_create (&(tid[1]), NULL, p, (void *)1);

    pthread_join (tid[0], NULL);
    pthread_join (tid[1], NULL);
    return 0;
}

```

- a) (0,5) Considerando as soluções para controle de acesso a seção crítica vistas em classe, qual delas é representada pelas funções void enterSC (int i) e void leaveSC (int i)?

Resposta: Solução de Peterson.

- b) (0,5) Destaque o trecho de código que utiliza espera ocupada na solução apresentada.

Resposta:

```

...
enterSC (int i)
{
...

```

```

while(flag[j] && turn == j);
...
}
...

```

- c) (0,5) Sob quais condições espera ocupada pode ser considerada uma solução adequada para o controle de acesso a seção crítica?

Resposta: Espera ocupada (busy wating) significa que um processo está esperando que uma condição seja satisfeita em um laço fechado sem liberar o processador. São úteis para sistemas multiprocessados onde uma *thread* executa em um laço de espera ocupada (busy-loop – por um curto intervalo de tempo), ao invés de incorrer na sobrecarga de ser colocada em uma fila de pronto (sleep).

- d) (0,5) Não está sendo usado controle de acesso a seção crítica no código acima, no contexto da função *p* que define o corpo da *thread* a ser criada. Considerando que a função *p* será uma seção crítica, como você faria para garantir o acesso exclusivo a essa região utilizando as funções `enterSC (int i)` e `leaveSC (int i)`?

Resposta:

```

void * p (void *arg){
    int id = (int *)arg;
    enterSC (id);
    if (id)
        printf("\nPj: A região crítica é minha!!!\n");
    else
        printf("\nPi: A região crítica é minha!!!\n");
    counter++;
    if (id)
        printf("\n\nPj: counter = %d\n\n", counter);
    else
        printf("\n\nPi: counter = %d\n\n", counter);
    leaveSC (id);
    return NULL;
}

```

9. (1,0) Considerando questões relacionadas a vinculação de endereços de instruções e dados a endereços de memória, as sentenças a seguir são verdadeiras, exceto:

(x) pode ocorrer vinculação em tempo de compilação, caso a posição de memória seja conhecida a priori (ex. em caso de acesso a endereços específicos de memória). Assim, código absoluto pode ser gerado, sem a necessidade de recompilação caso a posição de início mude;

() pode ocorrer vinculação em tempo de carga: nesse caso, deve-se gerar código realocável (*relocatable code*). Para tanto, considera-se que a posição de memória não é conhecida em tempo de compilação; e

() pode ocorrer vinculação em tempo de execução: nesse caso, a vinculação é atrasada até o tempo de execução. Ocorre se o processo pode ser movido durante a execução, de um segmento de

memória para outro. Nesse caso, existe a necessidade de suporte de hardware para mapas de endereço (ex., registradores base e limite).